SQL Server 2000 简单查询

1. 查询 products 表中 p_price（商品价格）在 800 以上的商品详细信息，SQL 代码如下所示：

```
USE eshop
SELECT *
FROM products
WHERE m_price > 800
```

2. 查询 products 表中 p_quantity（商品数量）在 20 和 50 之间的商品编号、商品名称和商品数量，SQL 代码如下所示：

```
USE eshop
SELECT p_no, p_name, p_quantity
FROM products
WHERE p_quantity >= 20
    AND p_quantity <= 50
```

或

```
USE eshop
SELECT p_no, p_name, p_quantity
FROM products
WHERE p_quantity BETWEEN 20 AND 50
```

3. 查询 orders 表中各会员购买商品的总量，并以汉字列标题形式输出会员帐号，商品总额，SQL 代码如下所示：

```
USE eshop
SELECT m_account  会员帐号, o_quantity  商品总额
FROM orders
```

4. 查询 members 表中家庭地址为“湖南”的会员详细信息，SQL 代码如下所示：

```
USE eshop
SELECT *
FROM members
WHERE m_address LIKE '湖南%'
```

5. 查询 members 表中年龄大于 30 且性别为“男”的会员详细信息，SQL 代码如下所示：

```
USE eshop
SELECT *
FROM members
WHERE    DATEDIFF(YY,m_birth,GETDATE())>30
    AND m_sex = '男'
```

6. 查询 orders 表各商品销售总量前 3 名的商品编号和销售总量，SQL 代码如下所示：

```
USE eshop
SELECT TOP 3 p_no, o_quantity
FROM orders
ORDER BY o_quantity DESC
```

7. 查询 orders 表中购买过商品的会员帐号，要求去掉重复行，SQL 代码如下所示：

```
USE eshop
SELECT DISTINCT m_account
FROM orders
```

8. 查询 orders 表已确认、已支付和已配送的订单详细信息，SQL 代码如下所示：

```
USE eshop
SELECT *
FROM orders
WHERE o_confirm_state = '1'
    AND o_pay_state = '1'
    AND o_send_state = '1'
```

SQL Server 2000 高级查询

1. 查询性别为“男”的会员详细信息，查询结果按月薪降序排列，SQL 代码如下所示：

```
USE eshop
SELECT *
FROM members
WHERE m_sex = '男'
ORDER BY m_salary DESC
```

2. 查询全体会员的会员帐号，姓名和年龄并按家庭地址升序排列，同一地址中的会员按年龄降序

www.51514B.COM

排
列，SQL 代码如下所示：

```
USE eshop
SELECT m_account, m_name, YEAR(GETDATE())-YEAR(m_birth)  年龄
FROM members
ORDER BY m_address, m_birth
```

或

```
USE eshop
SELECT m_account, m_name, DATEDIFF(YY, m_birth, GETDATE())  年龄
FROM members
ORDER BY m_address, m_birth
```

3. 查询会员帐号为'liuzc'所购买的商品号和订购日期，并按订购日期升序排列，SQL 代码如下所示：

```
USE eshop
SELECT p_no, o_date
FROM orders
WHERE m_account = 'liuzc'
ORDER BY o_date
```

4. 查询购买商品号为'0910810004'总人数，SQL 代码如下所示：

```
USE eshop
SELECT COUNT(*)
FROM orders
WHERE p_no = '0910810004'
```

5. 查询 2005 年 6 月 6 日前，所有商品的订购总量，要求输出商品号和订购总量，SQL 代码如下所示：

```
USE eshop
SELECT p_no, SUM(o_quantity)
FROM orders
WHERE o_date < '2005-6-6'
GROUP BY p_no
```

6. 查询所有会员的平均月薪，最高月薪和最低月薪之和，SQL 代码如下所示：

```
USE eshop
SELECT AVG(m_salary)+MAX(m_salary)+MIN(m_salary)
FROM members
```

7. 查询所有会员购买商品的种类和，要求输出会员号和商品种类和，SQL 代码如下所示：

```
USE eshop
SELECT m_account, COUNT(DISTINCT p_no)
FROM orders
GROUP BY m_account
```

8. 查询各类商品的最高购买数量，要求输出最高数量大于 10 的商品号和最高数量，SQL 代码如下所示：

```
USE eshop
SELECT TOP 1 p_no, SUM(o_quantity)
FROM orders
GROUP BY p_no
HAVING SUM(o_quantity) > 10
ORDER BY SUM(o_quantity) DESC
```

SQL Server 2000 联接查询

1. 查询购买了商品号为“0910810004”的会员号和姓名，并以汉字标题显示，SQL 代码如下所示：

```
USE eshop
SELECT DISTINCT members.m_account  会员号, m_name  姓名
FROM members
JOIN orders
ON members.m_account = orders.m_account
WHERE p_no = '0910810004'
```

2. 查询购买了商品名称为“爱国者 MP3”的会员号、姓名和商品价格，SQL 代码如下所示：

```
USE eshop
SELECT members.m_account, m_name,p_price
```

WWW.515148.COM

```
FROM members
JOIN orders
ON members.m_account = orders.m_account
JOIN products
ON orders.p_no = products.p_no
AND p_name = '爱国者 mp3'
```

3. 查询比“张自梁”月薪高的而和他不是同一地址的会员姓名和年龄，SQL 代码如下所示：

```
USE eshop
SELECT A.m_name,YEAR(GETDATE())-YEAR(A.m_birth)
FROM members A
JOIN members B
ON A.m_account <> B.m_account
AND B.m_name = '张自梁'
AND A.m_salary > B.m_salary
AND A.m_address <> B.m_address
```

4. 使用 exists 查询购买了“0910810004”商品的会员号和姓名，SQL 代码如下所示：

```
USE eshop
SELECT m_account, m_name
FROM members
WHERE EXISTS ( SELECT *
                    FROM orders
                    WHERE members.m_account = orders.m_account
                    AND p_no = '0910810004')
```

5. 使用 in 查询与“刘法治”购买至少同一种商品的会员号和商品号，SQL 代码如下所示：

```
USE eshop
SELECT DISTINCT A.m_account, A.p_no
FROM orders A
WHERE p_no IN ( SELECT p_no
                    FROM orders B
                    WHERE A.m_account <> B.m_account
                    AND B.m_account IN (SELECT m_account
                                        FROM members
                                        WHERE B.m_account = members.m_account
                                        AND m_name='刘法治'))
```

6. 使用简单查询家庭地址为“湖南株洲”的会员以及年龄在 30 岁以上的会员详细信息，SQL 代码如
下所示：

```
USE eshop
SELECT *
FROM members
WHERE m_address = '湖南株洲'

SELECT *
FROM members
WHERE (YEAR(GETDATE())-YEAR(m_birth)) > 30
```

再按 F5 或点击工具栏上的运行按钮“ ”，查看运行结果，如图 7-1 所示。

7. 使用集合查询家庭地址为“湖南株洲”的会员以及年龄在 30 岁以上的会员详细信息，并与步骤 6 进行比较，SQL 代码如下所示：

WWW.515148.COM

```
USE eshop
SELECT *
FROM members
WHERE m_address = '湖南株洲'
UNION
SELECT *
FROM members
WHERE (YEAR(GETDATE())-YEAR(m_birth)) > 30
```

再按 F5 或点击工具栏上的运行按钮“ ”，查看运行结果，如图 7-2 所示。

8. 将 members 表和 orders 表之间的左向外联接包括所有会员的信息，包括没有购买商品的会员，SQL 代码如下所示：

```
USE eshop
SELECT members.*, orders.*
FROM members
LEFT OUTER JOIN orders
ON members.m_account = orders.m_account
```

SQL Server 2000 的视图操作

1. 在 members 表中创建地址为“湖南株洲”的会员的视图 V_addr，SQL 代码如下所示：

```
CREATE VIEW V_addr
AS
    SELECT *
    FROM members
    WHERE m_address = '湖南株洲'
```

2. 在 orders 表中创建购买了商品号为“0910810004”商品的视图 V_buy，SQL 代码如下所示：

```
CREATE VIEW V_buy
AS
    SELECT *
    FROM orders
    WHERE p_no = '0910810004'
```

3. 在 members 和 orders 表上创建“湖南株洲”的会员购买了商品号为“0910810004”商品的视图 V_addr_buy，SQL 代码如下所示：

```
CREATE VIEW V_addr_buy
AS
    SELECT members.*
    FROM members
    JOIN orders
    ON members.m_account = orders.m_account
    AND p_no = '0910810004'
    AND m_address = '湖南株洲'
```

4. 在视图 V_addr 上查询性别为“男”的会员信息，SQL 代码如下所示：

```
USE eshop
SELECT *
FROM V_addr
WHERE m_sex = '男'
```

5. 在视图 V_addr 中增加一条记录（内容如下），并查看 members 表中记录的改变情况。记录内容如下：（T-SQL）

‘fengxk’，‘冯向克’，‘男’，‘1978-06-28’，‘北京市’，5000.0，‘fxk0628’

WWW.515148.COM

SQL 代码如下所示：

```
USE eshop
INSERT INTO V_addr VALUES('fengxk', '冯向克', '男', '1978-06-28', '北京市',5000.0, ' fxk0628')
```

6. 将视图 V_addr 中会员号为“liuzc518”的会员的密码修改为“liuzc0518”，并查看 members 中记录

的改变情况，SQL 代码如下所示：

```
USE eshop
UPDATE V_addr
SET m_password = 'liuzc0518'
WHERE m_password = 'liuzc518'
```

7. 在 V_addr 中删除会员号为“fengxk”的记录，并查看 members 中记录的改变情况，SQL 代码如下

所示：

```
USE eshop
DELETE
FROM V_addr
WHERE m_account = 'fengxk'
```

8. 删除视图 V_addr_buy、V_buy 和 V_addr，SQL 代码如下所示：

```
USE eshop
DROP VIEW V_addr_buy, V_buy, V_addr
```

学生表 S，课程表 C 和学生选课表 SC，它们的结构如下，试用 SQL 完成下列操作。

S（S#，SN，SEX，AGE，DEPT）
C（C#，CN，TEACHER）
SC（S#，C#，GRADE）

1. 建立数据库：

```
CREATE DATABASE student
ON PRIMARY
(
	NAME=student_dat,
	FILENAME='E:\mydata\student_dat.mdf',
	SIZE=10MB,
	MAXSIZE=50MB,
	FILEGROWTH=5MB
)
LOG ON
(
	NAME=student_log,
	FILENAME='E:\mydata\student_log.ldf',
	SIZE=10MB,
	MAXSIZE=50MB,
	FILEGROWTH=5MB
)
GO
```

2. 创建表

```
USE student
CREATE TABLE S
(
	S# VARCHAR(20),
	SN VARCHAR(20),
	SEX CHAR(2),
	AGE INT,
	DEPT VARCHAR(50)
)
GO
```

WWW.51514B.COM

```
USE student
CREATE TABLE C
(
	C# VARCHAR(20),
	CN VARCHAR(20),
	TEACHER VARCHAR(50)
)
GO

USE student
CREATE TABLE SC
(
	S# VARCHAR(20),
	C# VARCHAR(20),
	GRADE INT
)
GO

USE student
CREATE TABLE SC_C
(
	CNO VARCHAR(20),
	CNAME VARCHAR(20),
	AVG_GRADE INT
)
GO
```

⑴ 用 SQL 找出选修了“程军”老师教的所有课程的学生姓名。

```
SELECT DISTINCT SN FROM S
WHERE NOT EXISTS (SELECT * FROM C
                  WHERE C.TEACHER='程军' AND NOT EXISTS
                        (SELECT * FROM SC
                         WHERE
                           S.S#=SC.S#    AND SC.C#=C.C#))
GO
```

用 SQL 找出选修了“程军”老师教的课程的学生姓名。

```
SELECT DISTINCT SN FROM S,SC,C
WHERE S.S#=SC.S# AND SC.C#=C.C# AND TEACHER='程军'
GO
```

⑵ 用 SQL 找出“程序设计”课程成绩在 90 分以上的学生姓名。

```
SELECT DISTINCT SN FROM S,SC,C
WHERE S.S#=SC.S# AND SC.C#=C.C# AND CN='程序设计' AND GRADE>90
GO
```

⑶ 检索所有比“王华”年龄大的学生姓名、年龄和性别。

```
SELECT DISTINCT SN,AGE,SEX FROM S
WHERE AGE>(SELECT AGE FROM S
           WHERE SN='王华')
GO
```

⑷ 检索选修课程“C2”的学生中成绩最高的学生的学号。

```
SELECT S# FROM SC
WHERE C#='C2' AND GRADE>=ALL(SELECT GRADE FROM SC
                             WHERE C#='C2')
GO
```

或

```
SELECT S# FROM SC
WHERE C#='C2' AND GRADE=(SELECT MAX(GRADE) FROM SC
                         WHERE C#='C2')
GO
```

⑸ 检索学生姓名及其所选课程的课程号和成绩

```
SELECT SN,C#,GRADE FROM S,SC
WHERE S.S#=SC.S#
GO
```

⑹ 检索选修四门以上课程的学生总成绩（不统计不及格的课程），并要求按总成绩的降序排列出来。

WWW.515148.COM

```
SELECT SUM(GRADE) FROM SC
WHERE GRADE>=60
GROUP BY S# HAVING COUNT(*)>=4
ORDER BY SUM(GRADE) DESC
GO
```

⑺ 检索全是女同学选修的课程的课程号。

```
SELECT DISTINCT C# FROM SC,S
WHERE SC.S#=S.S# AND SEX='女'
GO
```

或

```
SELECT DISTINCT C# FROM SC
WHERE SC.S# IN (SELECT S.S# FROM S
                WHERE SEX='女')
GO
```

⑻ 检索不学 C6 课程的男同学的学号和姓名（S#，SN）。

```
SELECT S#,SN FROM S
WHERE SEX='男' AND S# NOT IN(SELECT S# FROM SC
                             WHERE C#='C6')
GO
```

⑼ 把 SC 表中每门课程的平均成绩插入到另一个已存在的表 SC_C(CNO,CNAME,AVG_GRADE)中。

```
INSERT INTO SC_C(CNO,CNAME,AVG_GRADE)
SELECT C.C#,CN,AVG(GRADE) FROM SC,C
WHERE SC.C#=C.C#
GROUP BY C.C#,CN
GO
```

⑽ 从 SC 表中把吴老师的女学生选课元组删除。

```
DELETE FROM SC
WHERE S# IN (SELECT S# FROM S
             WHERE SEX='女')
  AND C# IN (SELECT C# FROM C
             WHERE TEACHER LIKE '吴%')
GO
```

⑾ 从 SC 表中把数学课程中低于数学平均成绩的选课元组删去。

```
DELETE FROM SC
WHERE GRADE<(SELECT AVG(GRADE) FROM SC,C
             WHERE SC.C#=C.C# AND CN='数学'
             GROUP BY SC.C#)
GO
```

⑿ 把吴老师的女学生选课成绩增加 4%。

```
UPDATE SC
SET GRADE=GRADE+GRADE*0.04
WHERE S# IN(SELECT S# FROM S WHERE SEX='女')
   AND C# IN(SELECT C# FROM C WHERE TEACHER LIKE '吴%')
GO
```

课本知识点总结

P86 页 20 题

将下图的教务管理数据库的 E-R 图，转化为关系模型

WWW.515148.COM

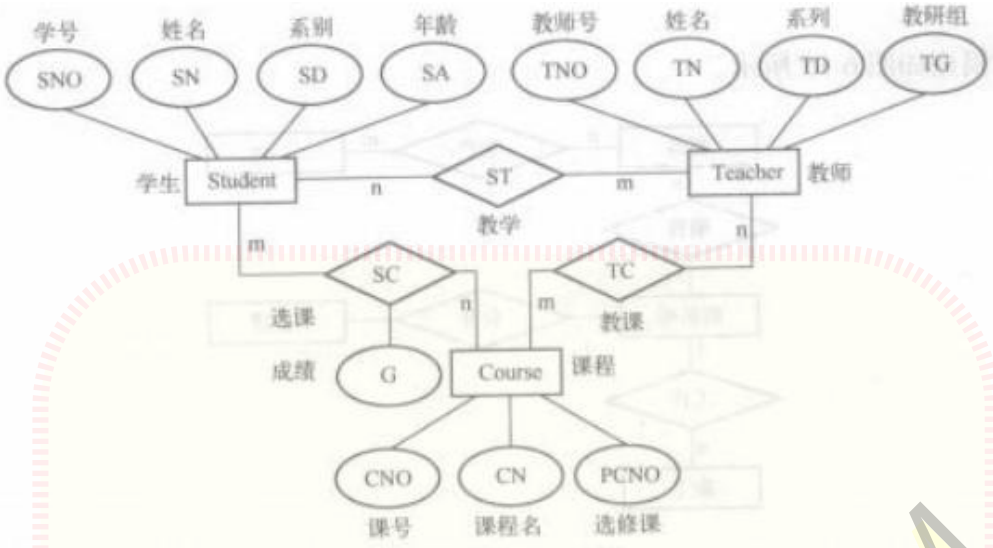


图 6-13　教学管理数据库

20. 图 6-13 是某个教务管理数据库的 E-R 图，请把它们转换为关系模型(图中关系、属性和联系的含义，已在它旁边用汉字标出)。

答：

转换后的关系模型如下：

Student(SNO,SN,SD,SA)；
Teacher(TNO,TN,TD,TG)；
Course(CNO,CN,PCNO)；
ST(SNO,TNO)；
SC(SNO,CNO,G)；
TC(TNO,CNO)。

P87 页 23 题

23. 设有如下实体。

学生：学号、单位、姓名、性别、年龄、选修课程名

课程：编号、课程名、开课单位、任课教师号

教师：教师号、姓名、性别、职称、讲授课程编号

单位：单位名称、电话、教师号、教师名

上述实体中存在如下联系：

1) 一个学生可选修多门课程，一门课程可为多个学生选修。

2) 一个教师可讲授多门课程，一门课程可为多个教师讲授。

3) 一个单位可有多个教师，一个教师只能属于一个单位。

试完成如下工作：

1) 分别设计学生选课和教师任课两个局部信息的结构 E-R 图。

2) 将上述设计完成的 E-R 图合并成一个全局 E-R 图。

3) 将该全局 E-R 图转换为等价的关系模型表示的数据库逻辑结构。

WWW.515148.COM

答：

1）学生选课局部 E-R 图如图 6-16 所示。

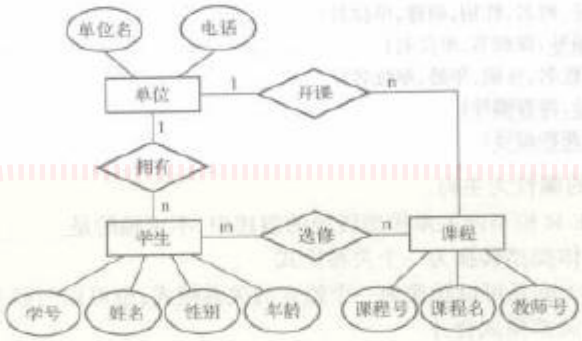


图 6-16　学生选课局部 E-R 图

教师任课局部 E-R 图如图 6-17 所示。



图 6-17　教师任课局部 E-R 图

2）合并的全局 E-R 图如图 6-18 所示。

为避免图形复杂，下面给出各实体属性：

单位：单位名、电话

学生：学号、姓名、性别、年龄

教师：教师号、姓名、性别、职称

课程：编号、课程名



图 6-18　合并的全局 E-R 图

P124 例 5—7 由学生、课程和选课 3 个表，定义一个计算机系的学生成绩视图，其属性包括学号、姓名、课程名和成绩

CREATE VIEW　学生成绩（学号，姓名，课程名，成绩）

AS SELECT 学生.学号，学生.姓名，课程.课程名，选课.成绩

FORM 学生，课程，选课

WHERE 学生.学号=选课. 学号 AND 课程.课程号=选课.课程号 AND 学生.所在系 ='计算机系'；

P124 例 5—8 将学生的学号、总成绩、平均成绩定义成一个视图

CREATE VIEW 学生成绩统计（学号，总成绩，平均成绩）

AS SELECT 学号，SUM（成绩），AVG(成绩)

WWW.515148.COM

FROM 选课

GROUP BY 学号;

P128 例 5—11 求选修 C1 课程的学生的学号和成绩，并要求对查询结果按成绩的降序排列，如果成绩相同，则按学号的升序排列

SELECT 学号，成绩

FROM 选课

WHERE 课程号=‘C1’

ORDER BY 成绩 DESC，学号 ASC;

P128 例 5—13 求数学系或计算机系姓张的学生的信息

SELECT *

FROM 学生

WHERE 所在系 IN (‘数学系’，‘计算机系’)AND 姓名 LIKE ‘张%’;

P212 习题 9

P199 例 6-30 设在读者表中增加了“借书册数”字段，现要求统计借书者在 1998 年~1999 年间所借书的册数，并将结果送入读者表中的借书册数字段。

UPDATE 读者
SET 借书册数=（SELECT COUNT(*)
FROM 读者，借阅
WHERE 借者.书证号=借阅.读者书证号 AND 借阅日期
BETWEEN ‘1998-01-01’AND ‘1999-12-31’）

P199 例 6-31 删除单位是计算机系的读者借阅记录。

DELETE 借阅
WHERE 读者书证号 IN(SELETE 书证号 FROM 读者 WHERE 单位=‘计算机系’)

在选课表中插入学号为 98011，**课程号为** c10，成绩为 90 的记录

```
insert
into 选课
values ('98011','c10',null);
insert
into 学生
values('10228','shenyim','20','na','jsj')


select distinct *
from 学生 where 姓名='shenyim'

update 学生
set 年龄='22'

update 学生
set 姓名='sdf'
where 学号='10228'


删除学生表
delete
from 学生


insert
into
学生
values('10220','sds','20','na','sjuj')
```

P239 习题 9

9. 现在要建立关于系、学生、班级、学会信息的一个关系数据库。语义为：一个系有若干专业，每个专业每年只招一个班，每个班有若干学生，一个系的学生住在同一个宿舍区，每个学

WWW.51518.COM

生可参加若干学会，每个学会有若干学生。

描述学生的属性有：学号、姓名、出生日期、系名、班号、宿舍区。

描述班级的属性有：班号、专业名、系名、人数、入校年份。

描述系的属性有：系名、系号、系办公室地点、人数。

描述学会的属性有：学会名、成立年份、地点、人数。学生参加某会有一个入会年份。

1）请写出关系模式。

2）写出每个关系模式的最小函数依赖集，指出是否存在传递依赖。在函数依赖左部是多属性的情况下，讨论函数依赖是完全依赖，还是部分函数依赖。

3）指出各个关系模式的候选码、外码，有没有全码？

答：

1）关系模式为：

学生(学号，姓名，出生日期，班级编码)；
班级(班级编码，专业名，系号，人数，入校年份)；
教学系(系名，系号，办公室地点，人数，宿舍区)；
学会(学会名，成立年份，地点，人数)；
参加(学号，学会名，入会年份)。

2）$F_{班级}$ = {班级编码→专业名，班级→系号，班级→人数，班级→入校年份}；

$F_{学生}$ = {学号→姓名，学号→出生日期，学号→班级编码}；

$F_{教学系}$ = {系号→系名，系号→办公室地点，系号→人数，系号→宿舍区}；

$F_{参加}$ = {学会名→成立年份，学会名→地点，学会名→人数}；

$F_{学会}$ = {(学号，学会名)→入会年份}。

3）学生表中，码为学号，外码为班级编码；

班级表中，码为班级编码，外码为系号；

教学系表中，码为系号；学会表中，码为学会名；

参加表中，码为(学号，学会名)，外码为学号和学会名，参照属性为学生(学号)和学会(学会名)。有候选码就无全码。

WWW.515148.COM